DeFuzz: Agentic Discovery of Silently-Failing Compiler Defenses via Cross-Mechanism Invariant Transfer

Anonymous Author(s)
Paper #XXX — IEEE S&P Double-Blind Review

Abstract—Compiler-inserted security defenses (e.g., stack canaries, CFI, and shadow stacks) serve as the last line of protection against memory-safety exploits. However, their implementations often fail to meet design expectations due to complex backend lowering and other root causes. These failures are silent: the program compiles cleanly and executes functionally correctly, but its security contract is broken. Finding silent defense failures is difficult due to two main challenges. First is the oracle gap; because there is no crash or semantic divergence, traditional fuzzing tools lack guidance. Second is the vast search space, as correctness must be validated across a two-dimensional matrix of defense mechanisms and Instruction Set Architectures (ISAs).

We present DeFuzz, an explicitly orchestrated agentic framework that systematically discovers silent defense failures. First, to bridge the oracle gap, we construct a deterministic oracle by extracting and formalizing defense semantics into machine-checkable security invariants. These invariants mandate that all verdicts derive from static binary states or runtime execution signals, eliminating reliance on non-deterministic Large Language Model (LLM) judgments. Second, to address the massive search space, we introduce a cross-mechanism invariant transfer pipeline that generalizes root causes from historical bugs into new invariants. Finally, because an invariant imposes different checking constraints under different ISAs, DeFuzz employs an explicitly orchestrated agentic loop that translates invariants into ISA-specific checkers and automates the discovery process. On GCC and LLVM, DeFuzz discovered [TBD: N] silent-failure defects, yielding [TBD: M] confirmed CVEs and upstream patches. These findings highlight the fragility of compiler defenses and demonstrate that explicitly orchestrated agent fuzzing, when strictly grounded, can systematically secure compiler backends.

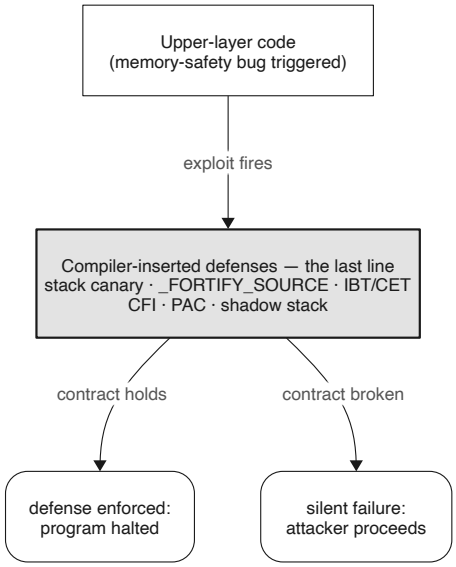


Fig. 1. Compiler security defenses as the runtime last line: they must hold once an upper-layer memory-safety bug is triggered.

I. Introduction

The number of publicly disclosed memory-safety vulnerabilities has grown relentlessly over the past decade. Major vendors and security researchers increasingly acknowledge that manual auditing and secure-coding disciplines alone are insufficient to eradicate memory-safety bugs in complex, upper-layer software. As a consequence, the behavior of a program after a vulnerability is triggered has become critical. Compiler security defenses—such as stack canaries, _FORTIFY_SOURCE, Control-Flow Integrity (CFI), Pointer Authentication (PAC), and shadow stacks—are deployed jointly by the compiler and the runtime environment as a fail-safe backstop intended to hold even when an exploit fires.

These defenses share a common operational model: the compiler analyzes the program's intermediate representations (e.g., AST, GIMPLE, or LLVM IR) and inserts check instructions, layout constraints, or metadata so that any runtime deviation is caught, halting the program. The stack canary is the canonical example—a random sentinel placed between a local buffer and the saved return address is verified before the function returns.

A. The Problem: Silent Defense Failures

However, defense mechanisms are themselves code, synthesized by complex compiler backends, and are susceptible to logic bugs. When a compiler defense fails, it often does so silently. CVE-2023-4039 exemplifies this phenomenon: the -fstack-protector mechanism in GCC

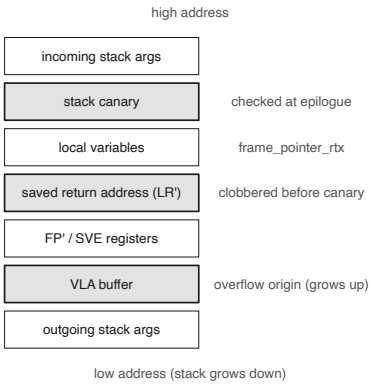


Fig. 2. CVE-2023-4039 buggy stack frame: canary above the VLA, return address below it, so an overflow bypasses the check.

was defeated on AArch64. When a function utilized a variable-length array (VLA), the backend placed the canary above the VLA, leaving the saved return address exposed below it. An overflow could thus rewrite the return address without disturbing the canary. Figure 4 illustrates this buggy frame layout.

This class of failure is exceptionally pernicious because it is double-silent: the toolchain issues no warnings during compilation, and the program executes its intended logic flawlessly at runtime. The security contract, however, is entirely broken, providing attackers with a complete bypass to an ostensibly protected binary. Such failures are driven by a variety of complex factors, including inconsistencies across Instruction Set Architectures (ISAs), mismatches between design and backend implementation, and incorrect assumptions regarding stack layouts. Note that while incorrect interactions between defense mechanisms and compilation optimization flags present an even larger search space, we currently scope our work to standard defense emissions.

B. Research Gap and Challenges

Despite the severity of silent defense failures, systematically exposing them remains an unsolved problem. Traditional testing and fuzzing methodologies (e.g., Csmith, YARPGen) rely on differential output or crash signals. Because silent failures preserve the original program semantics and do not induce crashes unless actively exploited, these tools register nothing. We term this fundamental lack of adjudicative capability the oracle gap.

Furthermore, the search space is massive. Compilers must uphold security contracts across a two-dimensional matrix of defense mechanisms $\times$ ISAs. Existing static analysis approaches struggle to generalize across this matrix because silent failures exhibit heterogeneous forms across different backends. Recent free-running LLM agents attempt to bridge this gap but suffer from severe hallucination, as they lack deterministic grounding when adjudicating complex binary outcomes.

C. Our Approach: DeFuzz

To address these gaps, we present DeFuzz, an agentic framework designed to systematically discover silent defense failures. Our key insight is that while the implementation details vary wildly across the defense matrix, the fundamental security invariants that govern them can be formalized and transferred. By decoupling the extraction of abstract invariants from the generation of ISA-specific execution checkers, we can automate the discovery of silent failures without relying on hallucinated model judgments.

DeFuzz operates in three main stages. First, we systematize the security invariants that defenses must satisfy into a machine-checkable oracle. To ensure rigor, our oracle strictly grounds every bug claim in deterministic binary states or runtime execution signals. Second, we introduce a cross-mechanism invariant transfer pipeline. This process utilizes confirmed historical bugs as probes to retrieve and generalize abstract root causes across diverse mechanisms. Finally, we orchestrate the discovery process using a deterministic agentic loop. Rather than allowing agents to dictate control flow, our orchestrator enforces a fixed pipeline (build $\rightarrow$ coverage $\rightarrow$ oracle), invoking agents only to generate seeds and provide semantic minimization feedback via a versioned blackboard.

D. Contributions

In summary, this paper makes the following contributions:

- A Deterministic Oracle for Silent Defense Failures. We construct a sound, machine-checkable oracle framework that bridges the silent-failure gap. By mandating that all verdicts stem from reproducible execution or static evidence, we eliminate the hallucination risks prevalent in naive LLM-driven vulnerability discovery (§VI).
- Cross-Mechanism Invariant Transfer. We propose a retrieval-augmented pipeline that distills historical bugs into abstract failure modes. By transferring these signatures across the mechanism $\times$ ISA matrix, we generated [DEFERRED: N] novel transferred invariants (§IV, §V).
- An Explicitly Orchestrated Agentic Framework. We design a fuzzing architecture where agents operate at fixed pipeline positions and communicate exclusively through a versioned blackboard. This explicitly orchestrated approach guarantees reproducibility and enables precise, ISA-bound checker routing (§VII).
- Empirical Discovery. Evaluating GCC and LLVM across multiple architectures, DeFuzz discovered [DEFERRED: N] previously unknown silent compiler

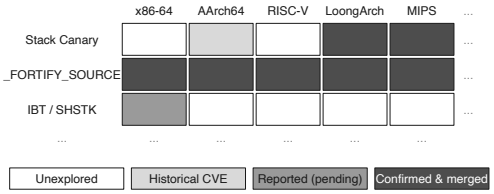


Fig. 3. The defense mechanism × ISA matrix. Each cell is an independent backend/pass; silent failures hide in individual cells.

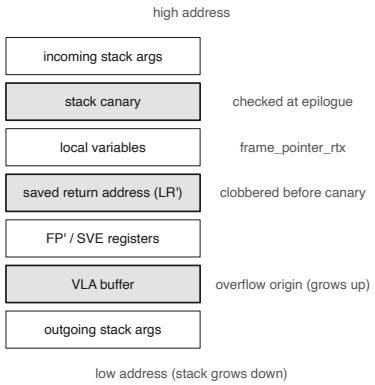


Fig. 4. CVE-2023-4039 buggy stack frame: canary above the VLA, return address below it, so an overflow bypasses the check.

defense failures, resulting in [DEFERRED: M] confirmed CVEs and upstream patches (§IX).

II. Background and Motivation

A. Compiler Security Defenses

A compiler security defense is a mechanism that inserts extra check instructions (e.g., canary comparisons, CFI target checks), layout constraints (e.g., guard placement, shadow-stack slots), or metadata (e.g., BTI landing pads, PAC signing) at compile time so that any run-time deviation is detected. These defenses are typically introduced during the middle-end IR phase (e.g., GIMPLE or LLVM IR) but must survive the transition to backend-specific representations (e.g., RTL, SelectionDAG, or Machine IR).

This lowering process is fraught with architecture-specific complexities. Register allocation, instruction scheduling, and stack frame finalization are dictated by the target ISA's Application Binary Interface (ABI). A defense mechanism that appears structurally sound in the target-independent IR can be silently dismantled during backend lowering. For instance, dynamic stack allocations (e.g., alloca or Variable Length Arrays) require target-specific frame pointer adjustments that can inadvertently overwrite or bypass previously established security boundaries. Consequently, the defense mechanism × ISA matrix has genuinely independent cells: a defense implemented correctly for one ISA may be silently broken on another due to disparate backend emission templates.

B. Threat Model and Scope

We evaluate the security of the compiler's emitted defense code under the following threat model, which aligns with the assumptions of the defense mechanisms themselves:

- Assumptions: The target application contains a latent, exploitable memory-safety vulnerability (e.g., a buffer overflow) in its upper-layer source code. The attacker possesses the capability to trigger this vulnerability and aims to hijack control flow or execute arbitrary code.
- In-Scope: We focus exclusively on the correctness and efficacy of the security mechanisms automatically inserted by the compiler. A vulnerability is considered in-scope if the defense mechanism fails to operate as expected (i.e., fails to detect or mitigate the exploit as designed). We describe only the violated security invariant (the falsification surface) and do not construct working end-to-end exploits.
- Out-of-Scope: Hardware-level vulnerabilities, side-channel attacks, business logic flaws, and explicit source-level bugs (e.g., intentionally bypassing defenses via inline assembly) are outside our threat model.

To maintain a stable reference for evidence grounding and reproduction, our empirical evaluation is pinned to GCC 16.1.

C. Silent Failure: A Representative Case

When a compiler defense fails, it often does so silently. The stack protector mechanism, for example, depends on the invariant "the canary lies between every overflow-reachable stack object and the saved return address." In CVE-2023-4039, when a function used a Variable Length Array (VLA), the AArch64 backend stack-frame construction logic placed the canary above the dynamically allocated area, while the saved return address resided below it. An overflow originating from the VLA thus overwrites the return address before the canary is ever evaluated.

This class of failure is double-silent: it raises no error at compile time (toolchain and functional tests pass) and produces correct functional results at run time (program behavior is nominally correct), yet the security contract is broken. Furthermore, homologous omissions frequently

persist. PR-96191, a sibling of CVE-2023-4039 fixed in 2020, covered primary architectures but left fallback backends untouched, allowing the same failure to persist silently on other targets for years.

D. Why Existing Methods Miss It

Systematically exposing silent failures requires overcoming the oracle gap. Crash and differential fuzzing (e.g., Csmith, YARPGen) monitor surface-level signals—whether the program crashes or disagrees with another compiler's output. By definition, a silent failure preserves the original program semantics and does not induce a crash unless actively exploited; thus, these tools register nothing. Static analysis is constrained by predefined patterns; silent failures exhibit heterogeneous forms across different backends, rendering pattern matching difficult to generalize. Finally, free-running agent fuzzers sacrifice reproducibility: when a Large Language Model (LLM) autonomously dictates control flow and bug adjudication, execution trajectories cannot be reliably replayed, ablations cannot be isolated, and bug claims cannot be rigorously traced back to deterministic evidence, leading to high false-positive rates due to LLM hallucination.

III. System Overview

DeFuzz separates offline knowledge construction from online bug search. The offline stage reads compiler source, specifications, and historical bug records. A segmented CoT review provides broad candidate extraction, while historical-bug RAG targets implementations that may share a known root cause. Candidates that survive evidence grounding and deduplication enter the invariant catalog and are implemented as static or dynamic checkers. Confirmed findings can be added to the historical-bug pool as new retrieval probes.

The online stage evaluates the compiler's defense emission by orchestrating the compilation and subsequent execution of generated seeds. At run initialization, the orchestrator enumerates the checker catalog into a fixed queue. Each target denotes one invariant–checker pair; checker metadata supplies the applicable ISAs. The target remains fixed until its round or wall-clock budget is exhausted. Within each round, the Generator produces a C seed and a hard-wired pipeline executes routing $\rightarrow$ build $\rightarrow$ coverage $\rightarrow$ oracle. Pass, NotApplicable, and Error enter the Feedback branch before the next round; Error remains visible as an infrastructure diagnostic and consumes the current round without becoming a bug. A Fail invokes the Minimizer and is rechecked against the same checker. All intermediate states and results are versioned for replay and audit.

Figure 6 shows one target-specific iteration end to end; the following sections detail each stage.

IV. Security Invariants for Compiler Defenses

A. What is a security invariant

A security invariant is a property a defense must satisfy for it to be effective; violating it means the mechanism has been silently weakened in the compiled artifact or at run time, whether or not the code itself contains an overt bug. Each invariant is recorded with a machine-verifiable statement and, separately, an observation field that names only the externally observable phenomenon of a violation—not how to detect it. The separation of the phenomenon from its detection recipe is deliberate: "the ___stack_chk_fail symbol is missing from the binary" is a phenomenon, whereas "look it up in the ELF .dynsym" is a detection mechanism that belongs to the oracle implementation.

B. Corpus Construction and Candidate Extraction

The invariant archive is synthesized from three primary knowledge domains: compiler source code (specifically comments and runtime assertions), official ABI specifications, and historical vulnerability reports alongside their corresponding patches. To ensure interoperability with downstream analysis and continuous integration pipelines, every extracted invariant is formalized into a uniform schema. This schema explicitly captures the target mechanism, the originating source evidence, and the observable phenomena of a violation.

Given the vast scale of the compiler codebase, relevant semantic evidence is frequently fragmented across disparate compilation phases and backend templates. To address this, we partition the source material by mechanism and function boundaries. We then apply a segmented Chain-of-Thought (CoT) prompting strategy [1], instructing the model to isolate the protected asset, the activation conditions, and the required security property within each segment. It is critical to note that this pass strictly produces candidates. To prevent hallucination and ensure empirical validity, every candidate must retain its provenance and subsequently pass the stringent grounding and novelty gates detailed in §V.

C. Machine-Checkable Form: Static vs. Dynamic

Each invariant is classified by whether its violation can be observed in the un-executed binary (e.g., symbols, sections, disassembly, strings) or if it strictly requires execution traces (e.g., exit status, output flags, register snapshots). This classification is guided by observability, decisiveness, computational cost, and attribution capability. To maintain precision, invariants exhibiting both static and dynamic violation signals are bifurcated into distinct checkers rather than being merged into a single heuristic.

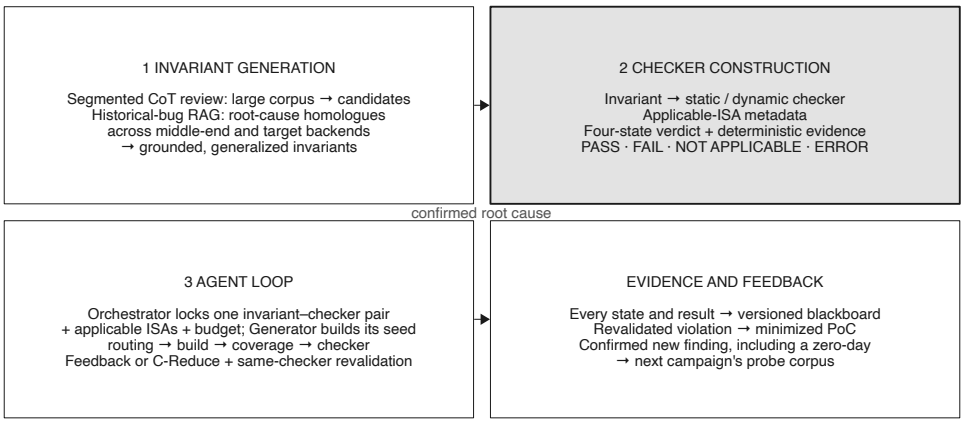


Fig. 5. DeFuzz technical route: corpus-grounded invariant generation feeds programmatic checkers, which define the targets of an explicitly orchestrated agent loop.

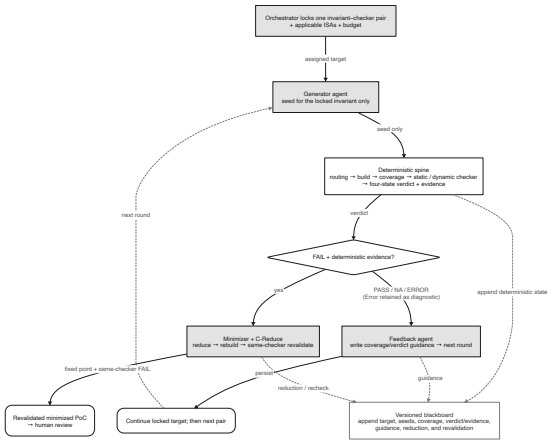


Fig. 6. Agent loop for one invariant–checker target and its applicable ISAs: generate → build → coverage → check → feedback or minimization.

V. Cross-Mechanism Invariant Transfer

A. Motivation: abstract-failure-mode transfer

Segmented review affords broad corpus coverage, yet it frequently fails to cluster isolated fragments that encode identical root causes, especially when the relevant evidence is fragmented across different mechanism names, compiler phases, or target backends. To bridge this gap, retrieval-augmented generation (RAG) [2] provides a complementary, high-yield path. By treating a confirmed historical bug as an explicit probe, the RAG phase actively searches for structurally related implementations. While this approach does not guarantee completeness, it significantly increases the probability of uncovering analogous defects that share a known root cause.

The RAG process begins by distilling a confirmed bug into a mechanism-agnostic failure signature. Retrieval then scans the indexed middle-end and backend corpora for implementations sharing similar semantic structures. We deliberately eschew entity-centric retrieval (e.g., matching API or symbol names), as it tends to confine results within the original defense mechanism. Instead, queries are formulated around abstract operations, violated assumptions, and protected assets, allowing a single probe to uncover analogous vulnerabilities across diverse mechanisms and backends.

B. Pipeline: Distillation, Analogy, Specialization, and Entailment

The generalization pipeline operates in four stages. First, a confirmed root cause undergoes distillation into an abstract description. For instance, using CVE-2023-4039 as a running example, the stack canary vulnerability is distilled from "AArch64 VLA overrides canary" into the abstract semantic signature "dynamically sized local allocations bypass statically computed security boundaries." Second, an analogy filter evaluates whether a retrieved code block embodies the same structural failure, proactively rejecting superficial lexical matches. Third, specialization instantiates the invariant within the concrete target source context (e.g., mapping the abstract boundary to specific shadow-stack frame calculations). Finally, entailment verifies that the resulting statement

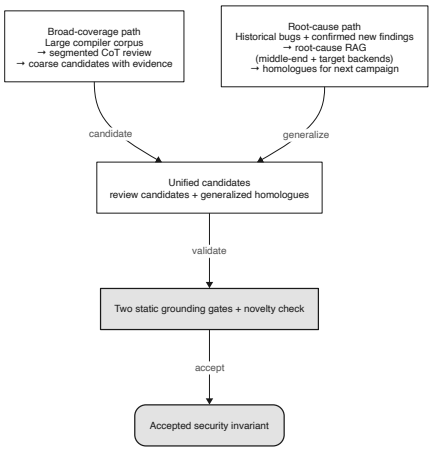


Fig. 7. Corpus-grounded invariant generation: segmented CoT review and historical-bug RAG feed a shared grounding and novelty filter; confirmed new findings return as retrieval probes.

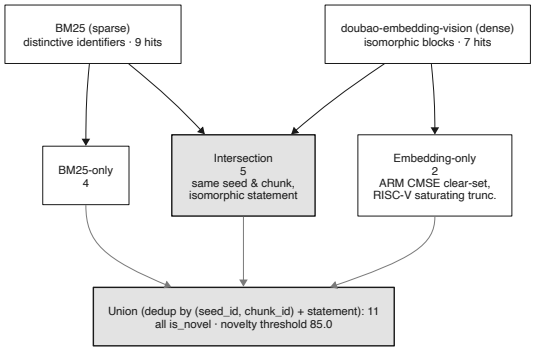


Fig. 8. Union/dedup of BM25 and embedding retrieval: 5 intersection, 4 BM25-only, 2 embedding-only, 11 union.

logically follows from the cited evidence. Figure 7 illustrates how segmented review and the RAG path converge into a shared grounding and novelty filter.

C. Retrieval: BM25 and embedding are complementary

To effectively query the codebase, each distilled root cause is synthesized into a dual-aspect query structure: a root_cause_phrase that provides an abstract semantic summary for dense embeddings, and critical_tokens that capture mechanism-specific identifiers (e.g., register names, macro flags) for sparse retrieval.

Within this hybrid RAG stage, we evaluate sparse BM25 and dense doubao-embedding-vision retrieval over the same 24 probe seeds and the same 4,496 GCC-16.1 source chunks. BM25 excels on the critical_tokens (ix86_zero_call_used_regs, SUBREG_PROMOTED, CONST_HIGH_PART); the dense retriever utilizes the root_cause_phrase to pull in semantically isomorphic blocks whose lexical anchors are weak (the CMSE clear-set constructor, the RISC-V saturating truncation). Deduplicating by (seed_id, chunk_id) and statement, the two paths yield 5 in the intersection, 4 BM25-only, and 2 embedding-only, for 11 in the union (Figure 8); all 11 are marked is_novel after a novelty check (threshold 85.0). Table I summarizes.

TABLE I
Transferred invariants by retrieval backend (24 probes, 4,496 GCC-16.1 chunks).

Category	Count	Note
Intersection (BM25 ∩ embed)	5	same seed & chunk, isomorphic statement
BM25-only	4	embed missed the block for that seed
Embedding-only	2	BM25 missed; dense retrieval added
Union (deduplicated)	11	$9 + 7 - 5$; all is_novel

D. Grounding gates and reproducibility

Candidates from both segmented CoT review and historical-bug RAG enter the same acceptance path. An invariant is accepted only after two static grounding gates confirm the cited evidence supports its statement and observation, and after the novelty check (threshold 85.0) confirms it does not duplicate an existing seed or manual rule. Because the dense retriever returns slightly different vectors per call, we cache query vectors keyed by query text so that top-k ordering—and hence the accepted set—reproduces across runs. After a new finding, including a zero-day, is reproduced and its root cause is confirmed, we serialize it in the same schema and add it to the probe pool. The next campaign can then search for further instances of that root cause.

VI. Oracle: From Invariants to Verdicts

A. Checker design

To operationalize an accepted invariant into an executable oracle, we define a formal checker contract. Let $\mathcal{P}$ be the space of compiled programs, $\mathcal{I}$ be the set of target ISAs, and $\mathcal{S}$ be the set of accepted invariants. An oracle is defined as a deterministic function:

$$\mathcal{O} : \mathcal{P} \times \mathcal{I} \times \mathcal{S} \to \{\text{Pass}, \text{Fail}, \text{NA}, \text{Error}\} \tag{1}$$

Invariants decidable without execution are implemented as static checkers analyzing symbols, sections, or disassembly. Conversely, invariants requiring runtime context (e.g., exit status, memory states) are realized as dynamic

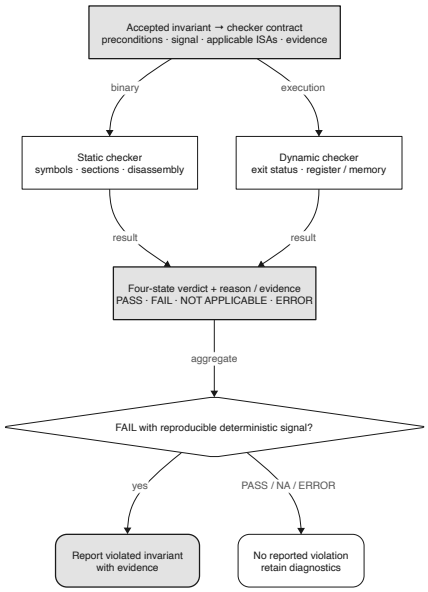


Fig. 9. An accepted invariant is translated into a static or dynamic checker; four-state verdicts pass through a deterministic-evidence gate before any finding is confirmed.

checkers. Invariants amenable to both are implemented redundantly, preserving deterministic precision over heuristic approximation.

Each checker returns a four-state verdict (pass / fail / not-applicable / error). NotApplicable is returned when the mechanism is off or the seed does not exercise it, and is ignored by the aggregation layer. A finding is counted as confirmed only when a deterministic checker fails or an execution differential reproduces, never on the strength of model output alone (Figure 9).

When a programmatic checker cannot decide an invariant–ISA cell, it returns NotApplicable or Error. The LLM is strictly prohibited from inferring or overriding verdicts based on disassembly or symbol evidence; all Fails must derive from deterministic execution or static analysis rules. Consequently, no provisional aggregate is ever promoted to violated by model output alone, ensuring that minimization and final bug claims are strictly grounded in reproducible evidence.

B. Mechanism aggregator and flag profiles

A per-mechanism aggregator combines checker verdicts: any Fail raises a mechanism-level violation carrying its evidence. On/off flag profiles let the oracle adjudicate differentially (defense enabled vs. disabled), and because the checkers key on binary and runtime signals rather than on the producing compiler, GCC and LLVM share the same checkers.

C. Checker metadata as a single source of truth

Each checker declares three metadata fields: which ISAs it binds, whether it is single-ISA or differential, and whether it is cheap or expensive. This metadata is the single source of truth read by both the gRPC and the MCP adapter, and it is what makes the ISA routing in §VII a table lookup rather than an agent decision.

VII. Agentic Loop

A. Design principles

The orchestrator, rather than an agent, owns target selection and control flow. At run initialization it constructs a fixed queue of invariant–checker pairs. For each target, checker metadata determines the applicable ISAs, and the target remains fixed for a configured round or time budget. Within a round, the pipeline follows the same order: generate → routing → build → coverage → oracle → route. Agents are invoked only at fixed positions and communicate through shared state. The Generator proposes; the oracle stack adjudicates, and confirmed findings require deterministic evidence.

B. Explicit orchestration and the blackboard

The agents form a closed loop where feedback guides subsequent generation, the oracle adjudicates outcomes, non-violations update feedback parameters, and violations trigger minimization. Crucially, all inter-agent communication flows through a versioned blackboard rather than direct dialogue (Figure 10). Algorithm 1 details this deterministic control flow.

Algorithm 1 Explicitly Orchestrated Agentic Loop

Require: Queue Q of invariant–checker targets, Budget B

Initialize Blackboard $\mathcal{B}$
for target $T \in Q$ do
  while budget $B > 0$ do
    $\mathcal{B}$.seed $\leftarrow$ Generator($\mathcal{B}$.feedback)
    $\mathcal{B}$.binary $\leftarrow$ Compile($\mathcal{B}$.seed, T.ISA)
    $v, \text{cov} \leftarrow \mathcal{O}(\mathcal{B}.\text{binary}, T.\text{ISA}, T.\text{inv})$
    if $v ==$ Fail then
      $\mathcal{B}$.poc $\leftarrow$ Minimizer($\mathcal{B}$.seed, $\mathcal{O}$)
      yield BugFound($\mathcal{B}$.poc)
    else if $v ==$ Pass $\vee$ $v ==$ NA then
      $\mathcal{B}$.feedback $\leftarrow$ FeedbackAgent(cov, v)
    end if
    $B \leftarrow B - 1$
  end while
end for

This architecture enforces reproducibility (as pinning a blackboard version deterministically fixes agent inputs), ablatability (enabling independent toggling of linkage edges), and auditability (ensuring every bug claim traces back to deterministic execution evidence).

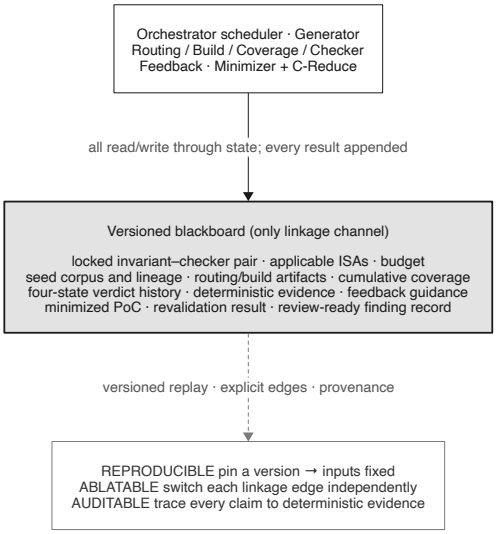


Fig. 10. The blackboard: all inter-agent linkage flows through versioned shared state, enabling reproduce / ablate / audit.

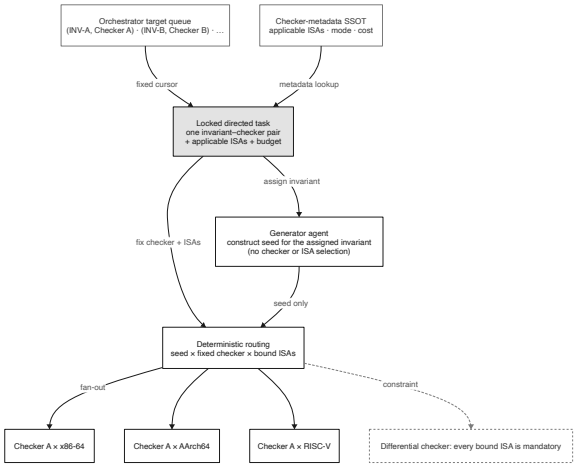


Fig. 11. Deterministic target scheduling: one invariant–checker pair remains fixed for its budget, while checker metadata expands the applicable ISAs and cheap static checkers remain enabled.

C. Three agents

The Generator (invoked at the start) carries read-only source-search and invariant-query tools and outputs a seed for the assigned invariant–checker pair; coverage is not an agent-reportable tool but is measured by the coverage node and fed back, so the agent cannot fabricate coverage. The feedback agent (invoked on a non-violation) is a context-isolated subagent that turns the coverage delta, checker identity, and the Pass/NotApplicable verdict into next-round guidance written back to the blackboard. The minimizer (invoked on a violation) shrinks the PoC with deterministic delta debugging (C-Reduce) as the workhorse and the LLM only for semantic guidance, so that reduction does not accidentally delete the structure that triggers the bug. Each candidate reduction is compiled and rechecked with the same checker before it is retained.

D. Checker-routed seeds, ISA-bound checkers

Each target in the deterministic queue denotes one invariant and its checker. The routing node expands that checker to every ISA in its metadata; the Generator never chooses either the target invariant or an ISA. A differential checker always expands to its full ISA set so that a cross-ISA signal cannot be pruned by model behavior. Cheap static checkers remain enabled as a side path, but they do not change the Generator's assigned target. When a checker returns Pass or NotApplicable, the Feedback agent records the relevant context and the same pair continues until its budget expires. Error follows the same non-bug path, remains visible as an infrastructure diagnostic, and consumes the current round. A Fail transfers the seed and deterministic evidence to the Minimizer. The blackboard records both branches before the orchestrator advances.

VIII. Implementation

The DeFuzz architecture is decoupled into a high-performance evaluation core and a Python-based agentic orchestrator. The core service, implemented in Go, provides deterministic build execution, coverage tracking, and oracle adjudication. It exposes these capabilities to the orchestrator via gRPC for high-throughput programmatic access, and via the Model Context Protocol (MCP) to provide agents with read-only tools for source code search, invariant querying, and delta debugging (C-Reduce) invocation. Crucially, the oracle definitions, static/dynamic checkers, and ISA applicability rules are centralized within the Go core, serving as the single source of truth for all metadata.

The orchestrator governs the explicit agentic loop, maintaining the strict topological order of generation, routing, build, and oracle adjudication. It manages the versioned blackboard, which tracks the lineage of seed corpora, cumulative coverage metrics, and feedback guidance. To ensure rigorous auditability, every execution persists a dedicated artifact bundle containing a serialized state chain, full toolchain configurations, and LLM hyperparameters. This design enables deterministic replay and post-hoc attribution of any confirmed bug claim.

IX. Evaluation

Setup. [DEFERRED: Awaiting experimental data. Setup must specify LLM parameters (model, tempera-

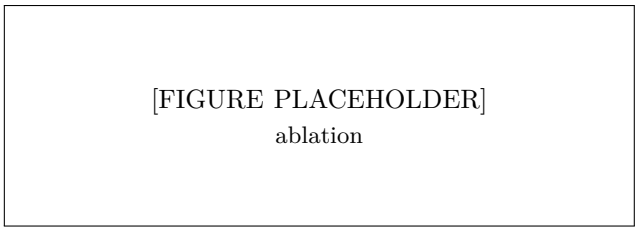


Fig. 12. Ablation: contribution of explicit orchestration (vs. free-running agents), oracle grounding, and coverage feedback to hit rate and cost.

ture), baseline configurations (free-running agent setup), and exact computational budget/hardware for reproducibility.]

a) RQ1 — Real bugs: [DEFERRED: Awaiting experimental data. Needs confirmed bug counts and CVE IDs.]

b) RQ2 — Invariant generation quality: [DEFERRED: Awaiting experimental data. Requires expert sampling grading (True Security Property, Reasonable but Non-essential, Trivial, Incorrect) and Cohen's κ calculation.]

c) RQ3 — Ablation: [DEFERRED: Awaiting experimental data. Requires evaluation of explicit orchestration vs. free-running baseline, coverage feedback, and oracle grounding.]

d) RQ4 — Cross-ISA generalization: [DEFERRED: Awaiting experimental data.]

X. Discussion and Limitations

[DEFERRED: Awaiting experimental data. Must include: (1) Manual effort analysis for converting invariants to checkers, (2) LLM API token cost/budget analysis, (3) False positive analysis, specifically regarding QEMU simulation fidelity vs. bare-metal hardware.]

XI. Related Work

Compiler Bug Finding and The Oracle Gap. Traditional compiler fuzzing has achieved immense success in uncovering optimization bugs and crashes. Tools like Csmith [3], YARPGen [4], and EMI [5] rely fundamentally on differential testing or crash detection as their oracle. However, these methods possess a structural blind spot regarding security defenses. A silent failure, by definition, preserves the functional semantics of the source code; the compiled artifact executes correctly and its output matches that of unoptimized or cross-compiler builds. Because there is no semantic divergence or runtime crash, differential and crash-based oracles are theoretically incapable of detecting these security contract violations.

Silent and Security Bugs in Compilers. The recognition of silent security failures in compilers is gaining traction. Studies such as "Silent Bugs Matter" [6] provide crucial empirical taxonomies of compiler-introduced security bugs, confirming that optimizations can inadvertently eliminate security checks or introduce timing side-channels. However, these works are primarily measurement studies and do not provide an automated, generative system for detecting new instances of such failures across different backends. DeFuzz bridges this gap by operationalizing these observations into an executable oracle.

LLM and RAG-based Property Generation. Recent advancements leverage Large Language Models to deduce system properties. PropertyGPT [7] employs Retrieval-Augmented Generation to synthesize smart-contract invariants, while SpecAuditor [8] identifies specification violations through semantic generalization. A critical distinction in DeFuzz is our retrieval strategy. Existing methods predominantly rely on entity-centric retrieval (e.g., matching specific API or variable names). In contrast, DeFuzz deliberately rejects entity-driven matching in favor of abstract-failure-mode transfer (§V), utilizing dual-aspect queries to bridge the lexical divide between disparate defense mechanisms (e.g., stack canaries vs. PAC) and architectures.

LLM-Agent Fuzzing: Hallucination vs. Determinism. The integration of LLMs as autonomous agents in fuzzing pipelines (e.g., AgentFuzz [9]) introduces powerful directed exploration capabilities. However, free-running agent architectures suffer from severe reproducibility and hallucination issues. When an LLM is granted the autonomy to decide whether a complex output constitutes a "bug," the system inevitably yields false positives driven by model hallucination. DeFuzz structurally diverges from this paradigm through explicit orchestration (§VII). By confining the LLM to generation and feedback roles, and strictly mandating that all bug claims be grounded in the deterministic execution of programmatic checkers, DeFuzz eliminates hallucination from the adjudication phase, ensuring that every reported violation is a reproducible, actionable defect.

XII. Conclusion

Compiler-inserted security defenses represent a critical backstop against memory-safety exploits, yet their efficacy is frequently compromised by silent failures during backend lowering. These failures—which bypass traditional crash and differential fuzzing oracles—leave binaries seemingly protected but functionally vulnerable across the mechanism $\times$ ISA matrix.

DeFuzz addresses this fundamental blind spot. By distilling defense semantics into a deterministic, machine-checkable oracle, we transition the detection of silent failures from ad-hoc manual auditing to a systematic, reproducible science. Our cross-mechanism invariant transfer pipeline demonstrates that historical root causes can be abstracted and transferred to uncover analogous flaws in disparate defenses. Furthermore, our explicitly orchestrated agentic loop proves that LLMs can be effectively harnessed for complex fuzzing tasks without succumbing

to adjudication hallucinations, provided their outputs are strictly grounded in deterministic execution evidence.

Ultimately, DeFuzz discovered [DEFERRED: N] previously unknown silent failures in production compilers, resulting in [DEFERRED: M] CVEs. These findings underscore not only the fragility of current compiler defenses but also the necessity of integrating rigorous, invariant-grounded oracles into the compiler development lifecycle.

References

[1] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in Advances in Neural Information Processing Systems, vol. 35, 2022, pp. 24 824–24 837.

[2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 9459–9474.

[3] X. Yang, Y. Chen, E. Eide, and J. Regehr, "Finding and understanding bugs in C compilers," in Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), 2011.

[4] V. Livinskii, D. Babokin, and J. Regehr, "Random testing for C and C++ compilers with YARPGen," in Proceedings of the ACM on Programming Languages (OOPSLA), 2020.

[5] V. Le, M. Afshari, and Z. Su, "Compiler validation via equivalence modulo inputs," in Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), 2014.

[6] G. A. Di Luna et al., "Silent bugs matter: A study of compiler-introduced security bugs," in Proceedings of the USENIX Security Symposium, 2023.

[7] Y. Liu, Y. Xue, D. Wu, Y. Sun, Y. Li, M. Shi, and Y. Liu, "PropertyGPT: LLM-driven formal verification of smart contracts through retrieval-augmented property generation," in Proceedings of the Network and Distributed System Security Symposium (NDSS), 2025.

[8] M. Lin and H. Chen, "SpecAuditor: Detecting specification violations via semantic generalization," in IEEE Symposium on Security and Privacy (S&P), 2026.

[9] F. Liu et al., "Testing the robustness of LLM-based agents via directed greybox fuzzing," in Proceedings of the USENIX Security Symposium, 2025.